22AIE205

Object-Oriented Programming & Exception Handling in Python

classes, objects, inheritance, polymorphism, and robust error handling with try / except

Dr. Prabu M | Dept. of Computer Science and Engineering | Amrita School of Computing, Chennai

Today's Roadmap

1

Part 1 — OOP Foundations

- Classes, objects & the blueprint analogy
- __init__, self, attributes
- Instance vs. class attributes & methods
- Encapsulation
- Inheritance & super()
- Polymorphism
- Magic / dunder methods

2

Part 2 — Exception Handling

- Errors: syntax vs. runtime vs. logical
- Common built-in exceptions
- try / except basics
- Handling multiple exception types
- else and finally
- raise & custom exceptions
- Live coding: safe input validator

Why Object-Oriented Programming?

Modelling the real world in code

1

State

Real-world things carry data — a car has a speed, colour, fuel level.

2

Behaviour

Real-world things do things — a car can accelerate, brake, refuel.

3

Bundling

OOP bundles state + behaviour together into one reusable unit: an object.

```
# Without OOP: data and logic are separate
name = "Ferrari"
speed = 0

def accelerate(speed, amount):
  return speed + amount

speed = accelerate(speed, 40)

# With OOP: bundled into one object
class Car:
  def __init__(self, name):
    self.name = name
    self.speed = 0

  def accelerate(self, amount):
    self.speed += amount
```

Classes & Objects

A class is a blueprint. An object is a specific thing built from it.

Blueprint (Class)

Defines what every house will have: number of rooms, colour, doors — and what it can do: open_door(), lock().

```
class House:
def __init__(self, rooms, colour):
    self.rooms = rooms
    self.colour = colour

def open_door(self):
    print("Door opened")
```

Objects (Instances)

Each house built from the blueprint is independent — its own rooms and colour.

```
h1 = House(3, "White")
h2 = House(5, "Blue")

h1.open_door()   # Door opened
print(h1.rooms)  # 3
print(h2.rooms)  # 5   <- independent!
```

Defining a Class in Python

Syntax anatomy

```
class Dog:
  species = "Canine"          # class attribute

  def __init__(self, name, age):    # constructor
    self.name = name         # instance attribute
    self.age = age

  def bark(self):                    # instance
method
    print(f"{self.name} says Woof!")

buddy = Dog("Buddy", 3)   # creating an object
buddy.bark()               # Buddy says Woof!
```

`class Dog:`

Keyword class + name in PascalCase, then a colon.

`__init__`

Runs automatically when an object is created — sets up initial state.

`self`

Refers to the specific object being created/used.

`buddy = Dog(...)`

Calling the class creates ("instantiates") an object.

Instance Attributes vs. Class Attributes

Per-object data vs. shared data

Instance Attributes

Belong to one specific object. Set inside __init__ using self.attr — every object gets its own copy.

```
class Dog:
  def __init__(self, name):
    self.name = name

d1 = Dog("Rex")
d2 = Dog("Milo")
d1.name   # "Rex"
d2.name   # "Milo"
```

Class Attributes

Shared by every object of the class. Defined directly inside the class body, outside any method.

```
class Dog:
  species = "Canine"  # shared
  def __init__(self, name):
    self.name = name

d1 = Dog("Rex"); d2 = Dog("Milo")
d1.species  # "Canine"
d2.species  # "Canine" (same!)
```

Understanding self

Why every method needs it — and why you never pass it explicitly

```
class Counter:
  def __init__(self):
    self.count = 0

  def increment(self):
    self.count += 1

c1 = Counter()
c2 = Counter()

c1.increment()      # Python runs:
                    # Counter.increment(c1)
print(c1.count)     # 1
print(c2.count)     # 0  <- untouched
```

- self is simply the object the method is being called on.
- Python passes it automatically: obj.method(args) becomes Class.method(obj, args) behind the scenes.
- It must be the first parameter of every instance method — but you never supply it yourself when calling.
- Naming it "self" is convention, not a keyword — but never deviate; every Python programmer expects it.

Encapsulation

Controlling access to an object's internal data

Naming convention signals intended access level:

public

`name`

Accessible from anywhere — the default.

_protected

`_balance`

Convention only: "internal use" hint, still accessible.

__private

`__pin`

Name-mangled to _Class__pin — discourages outside access.

```
class Account:
  def __init__(self, bal):
    self.__balance = bal

  def deposit(self, amt):
    self.__balance += amt

  def get_balance(self):
    return self.__balance

acc = Account(1000)
acc.deposit(500)
acc.get_balance()   # 1500
acc.__balance       # AttributeError
```

Live Coding — Stage Question 1

Build a BankAccount class

- __init__(self, owner, balance=0) — stores owner name and starting balance
- deposit(self, amount) — adds to balance
- withdraw(self, amount) — subtracts from balance, but must not allow balance to go negative
- __str__(self) — returns a friendly summary, e.g. ”Arun account: ₹5000"

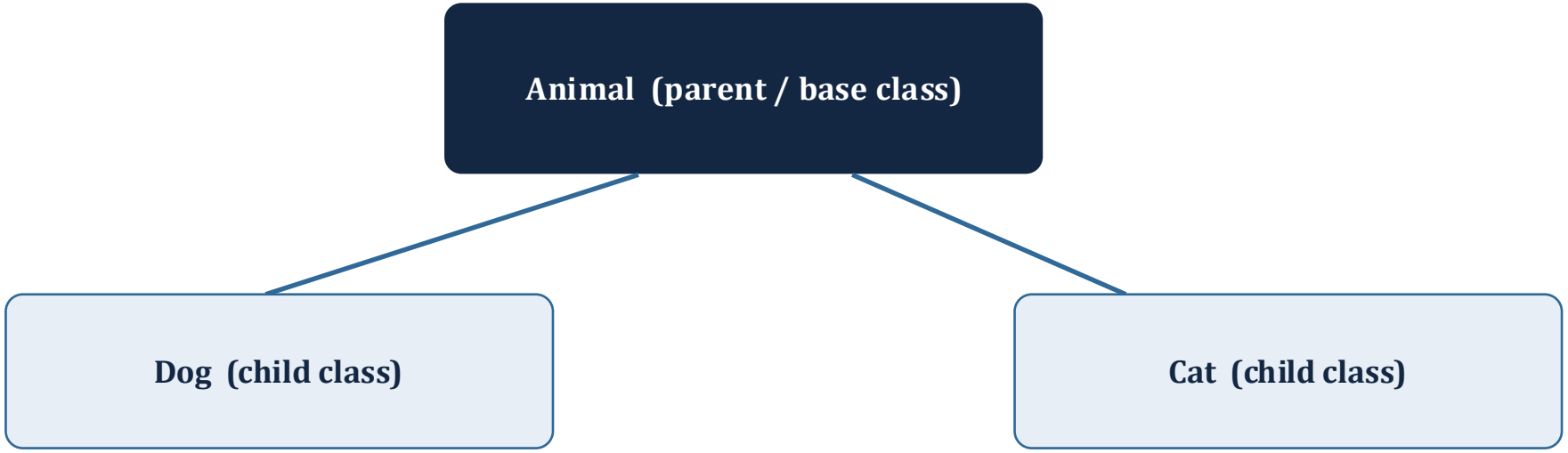
```
class BankAccount:
  def __init__(self, owner,
    balance=0):
    self.owner = owner
    self.balance = balance

  def deposit(self, amount):
    # your code here

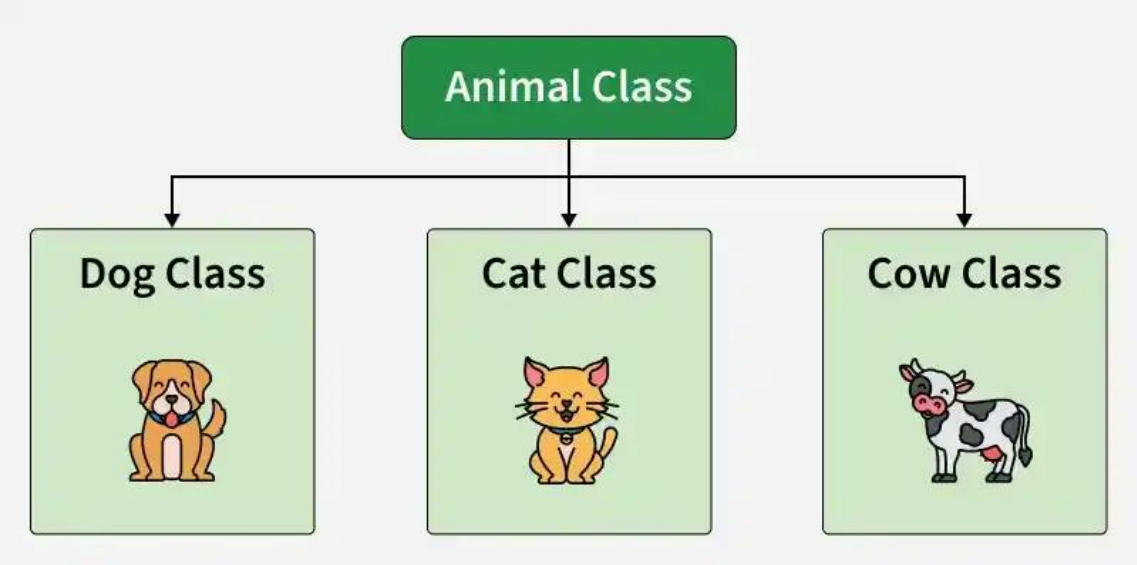
  def withdraw(self, amount):
    # your code here
```

Inheritance

Reuse a class's code by building an "is-a" relationship



- A Dog "is an" Animal — it should inherit everything an Animal already has.
- The child class gets all attributes and methods of the parent, and can add or override its own.
- Avoids duplicating code across similar classes.



Benefits Of Inheritance

•Promotes code reusability by sharing attributes and methods across classes.

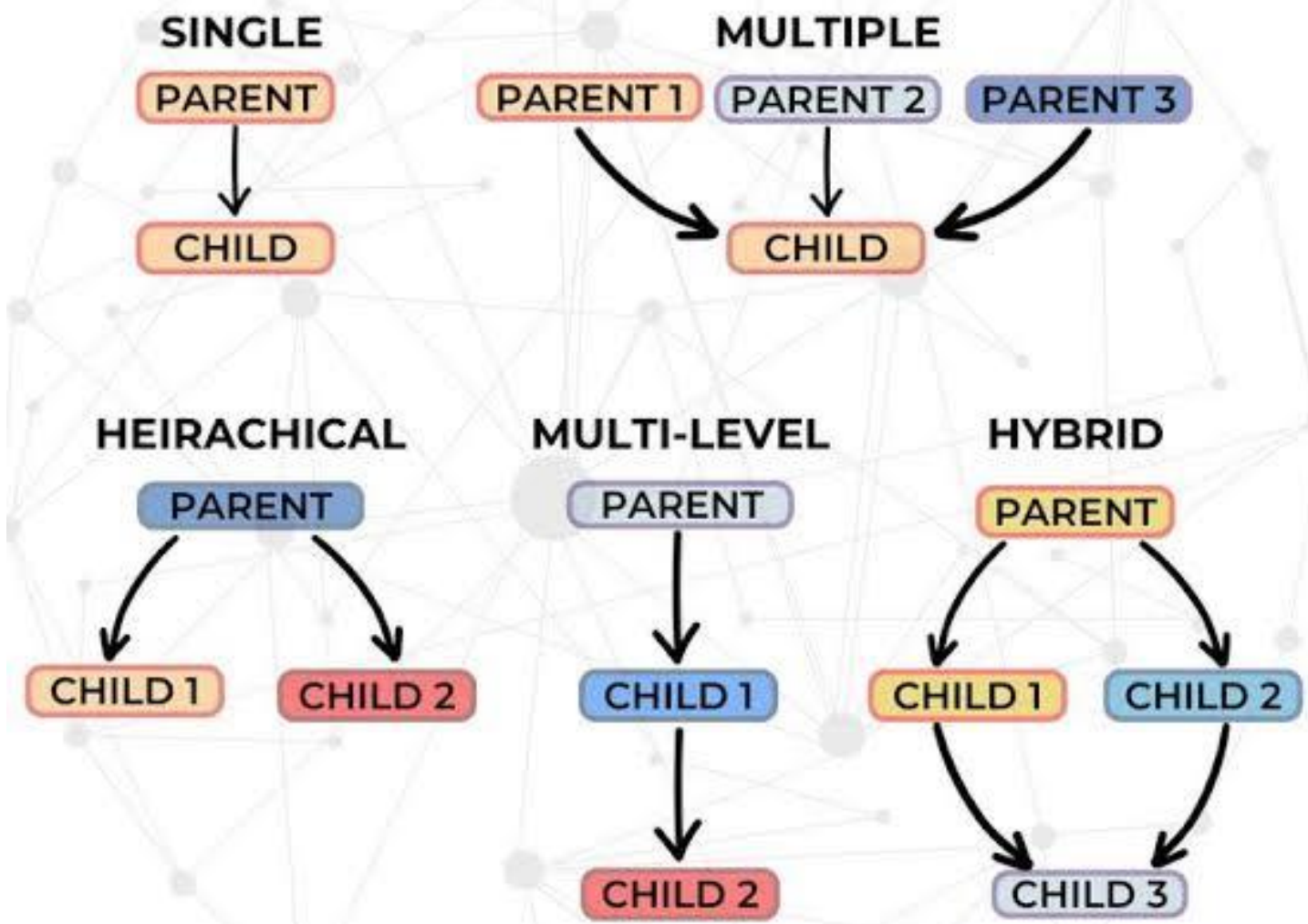
•Models real-world hierarchies like Animal -> Dog or Person -> Employee.

•Simplifies maintenance through centralized updates in parent classes.

•Enables method overriding for customized subclass behavior.

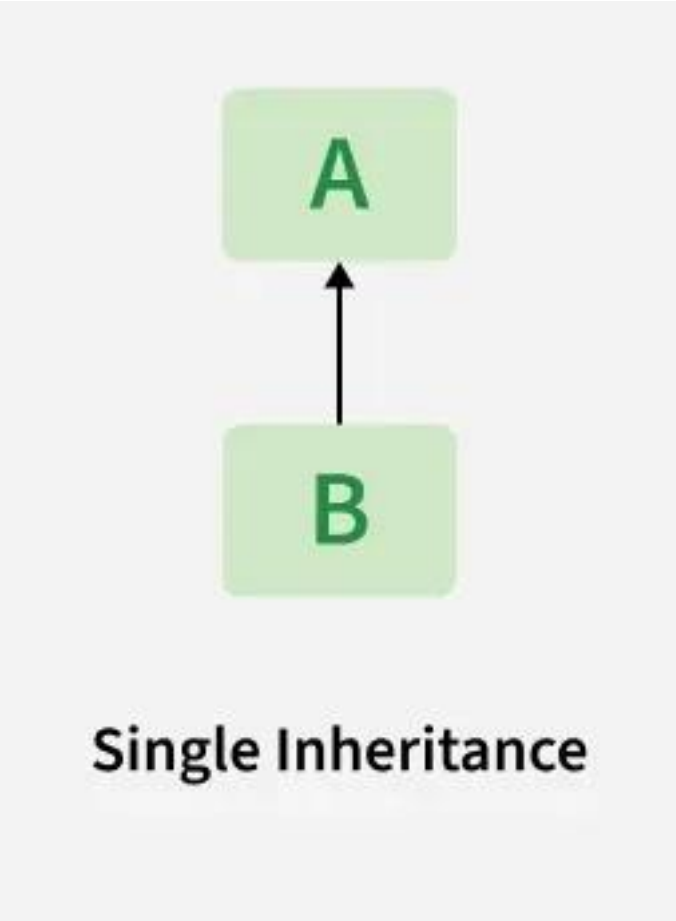
•Supports scalable, extensible design using polymorphism.

INHERITANCE TYPES



Single Inheritance

Single inheritance **enables a derived class to inherit properties from a single parent class**, thus enabling code reusability and the addition of new features to existing code.



Single Inheritance

Example:

```
# Base class
class Parent:
    def func1(self):
        print("This function is in parent class.")

# Derived class
class Child(Parent):
    def func2(self):
        print("This function is in child class.")

# Driver code
obj = Child()
obj.func1()
obj.func2()
```

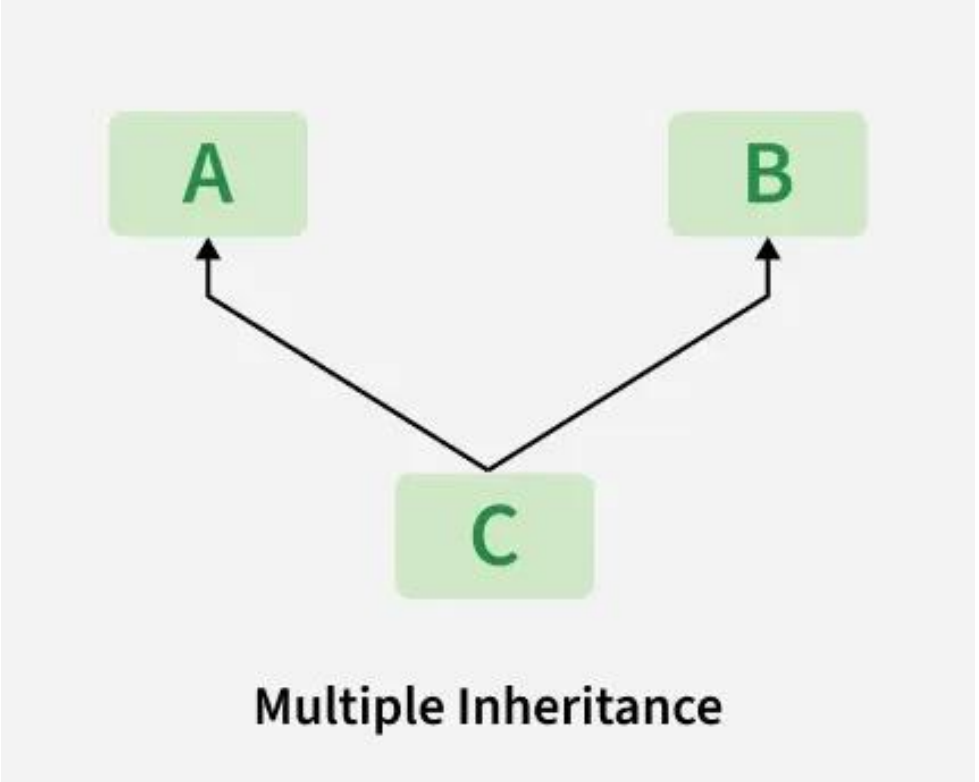
Output

```
This function is in parent class.
This function is in child class.
```

Multiple Inheritance

When a class can be derived from more than one base class this type of inheritance is called multiple inheritances.

In multiple inheritances, all the features of the base classes are inherited into the derived class.



Multiple Inheritance

Example:

```
# Base class 1
class Mother:
    mothername = ""

    def mother(self):
        print(self.mothername)

# Base class 2
class Father:
    fathername = ""

    def father(self):
        print(self.fathername)

# Derived class
class Son(Mother, Father):
    def parents(self):
        print("Father :", self.fathername)
        print("Mother :", self.mothername)
```

```
# Driver code
s1 = Son()
s1.fathername = "RAM"
s1.mothername = "SITA"
s1.parents()
```

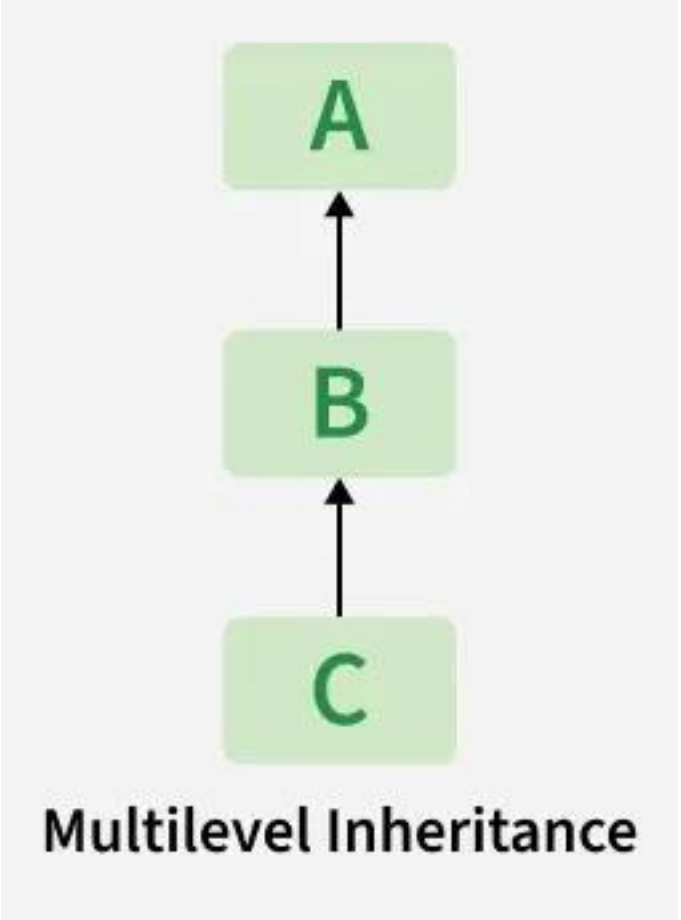
Output

```
Father : RAM
Mother : SITA
```

Multilevel Inheritance

In multilevel inheritance, features of the base class and the derived class are further inherited into the new derived class.

This is similar to a relationship representing a child and a grandfather.



Multilevel Inheritance

Example:

```
# Base class
class Grandfather:
    def __init__(self, grandfathername):
        self.grandfathername = grandfathername

# Intermediate class
class Father(Grandfather):
    def __init__(self, fathername, grandfathername):
        self.fathername = fathername
        # Call the constructor of Grandfather
        Grandfather.__init__(self, grandfathername)

# Derived class
class Son(Father):
    def __init__(self, sonname, fathername, grandfathername):
        self.sonname = sonname
        # Call the constructor of Father
        Father.__init__(self, fathername, grandfathername)

    def print_name(self):
        print('Grandfather name :', self.grandfathername)
        print('Father name :', self.fathername)
        print('Son name :', self.sonname)
```

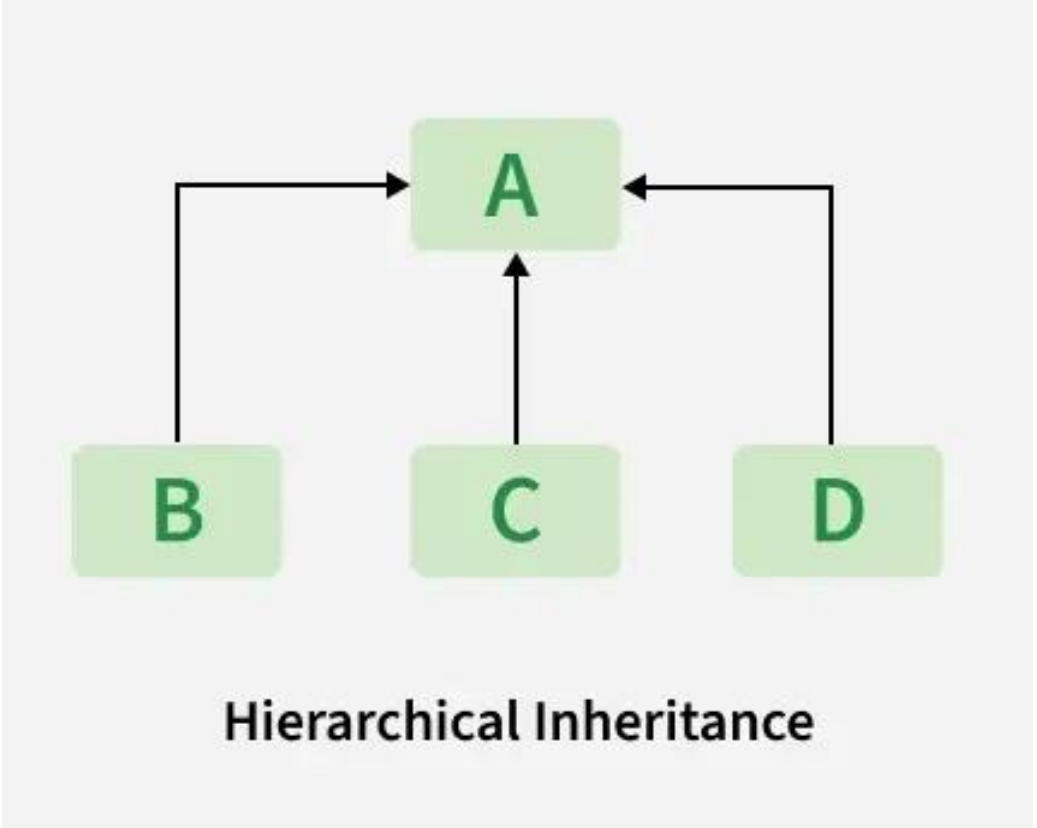
```
# Driver code
s1 = Son('Prince', 'Rampal', 'Lal mani')
print(s1.grandfathername)
s1.print_name()
```

Output

```
Lal mani
Grandfather name : Lal mani
Father name : Rampal
Son name : Prince
```

Hierarchical Inheritance

When more than one derived class are created from a single base this type of inheritance is called hierarchical inheritance.



Hierarchical Inheritance

```
# Base class
class Parent:
    def func1(self):
        print("This function is in parent class.")

# Derived class 1
class Child1(Parent):
    def func2(self):
        print("This function is in child 1.")

# Derived class 2
class Child2(Parent):
    def func3(self):
        print("This function is in child 2.")

# Driver code
object1 = Child1()
object2 = Child2()

object1.func1()
object1.func2()
object2.func1()
object2.func3()
```

Output

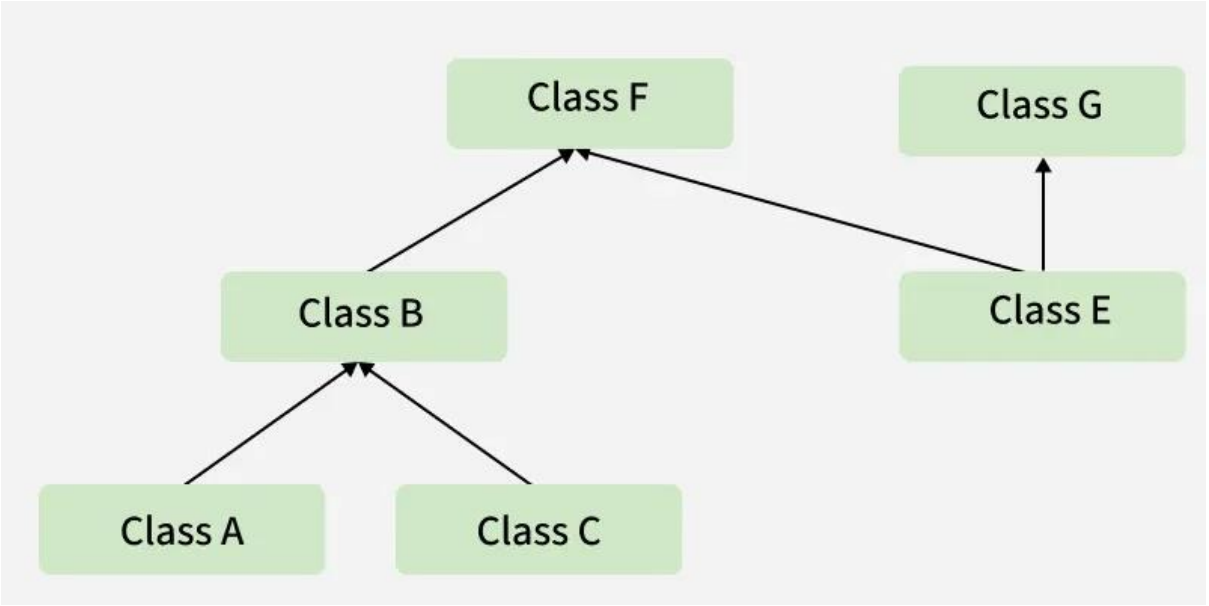
```
This function is in parent class.
This function is in child 1.
This function is in parent class.
This function is in child 2.
```

Hybrid Inheritance

Hybrid inheritance is a combination of more than one type of inheritance.

It uses a mix like single, multiple, or multilevel inheritance within the same program.

Python's method resolution order (MRO) handles such cases.



Hybrid Inheritance

```
# Base class
class School:
    def func1(self):
        print("This function is in school.")

# Derived class 1 (Single Inheritance)
class Student1(School):
    def func2(self):
        print("This function is in student 1.")

# Derived class 2 (Another Single Inheritance)
class Student2(School):
    def func3(self):
        print("This function is in student 2.")

# Derived class 3 (Multiple Inheritance)
class Student3(Student1, School):
    def func4(self):
        print("This function is in student 3.")

# Driver code
obj = Student3()
obj.func1()
obj.func2()
```

Output

```
This function is in school.
This function is in student 1.
```

Inheritance in Code

The child class lists the parent in parentheses

```
class Animal:
  def __init__(self, name):
    self.name = name

  def speak(self):
    print(f"{self.name} makes a sound")

class Dog(Animal):        # inherits from Animal
  def fetch(self):
    print(f"{self.name} fetches the toy")

d = Dog("Rex")
d.speak()   # inherited from Animal
d.fetch()   # defined in Dog
```

`class Dog(Animal):`

Parenthesised name is the parent class.

`d.speak()`

Dog has no speak() of its own — Python looks up the parent.

`d.fetch()`

Only Dog objects can fetch(); Animal objects can't.

Method Overriding

A child class can redefine a parent's method — and still reuse it

```
class Animal:
    def display(self):
        print("This is an animal")

class Dog(Animal):
    def display(self):
        print("This is a dog")

obj = Dog()
obj.display()
```

Output

```
This is a dog
```

- Overriding: a child method with the same name replaces the parent's version.
- super() gives access to the parent class's version of a method — use it to extend, not just replace, behaviour.
- Very common inside __init__ to avoid re-writing attribute setup.

Method Overriding & super()

A child class can redefine a parent's method — and still reuse it

When a child class defines its own constructor (__init__()), the parent class constructor is not called automatically.

We can use super().__init__() to execute the parent constructor and initialize any attributes defined there before adding child-specific behavior.

Example: In this example, the child class overrides both the constructor and a method inherited from the parent class.

```
class Employee:
    def __init__(self):
        self.role = "Employee"

    def display(self):
        print("Role:", self.role)

class Manager(Employee):
    def __init__(self):
        super().__init__()
        self.role = "Manager"

    def display(self):
        print("Role:", self.role)

e1 = Employee()
e2 = Manager()
e1.display()
e2.display()
```

Output

```
Role: Employee
Role: Manager
```

Overriding in Multiple Inheritance

The child class overrides one method and inherits another unchanged.

```
class Teacher:
    def introduce(self):
        print("I am a Teacher")

class Writer:
    def write(self):
        print("Writing an article")

class Author(Teacher, Writer):
    def introduce(self):
        print("I am an Author")

obj = Author()
obj.introduce()
obj.write()
```

Output

```
I am an Author
Writing an article
```

Overriding in Multilevel Inheritance

The grandchild class overrides a method inherited through multiple levels.

```
class Vehicle:
    def start(self):
        print("Vehicle started")

class Car(Vehicle):
    def start(self):
        print("Car started")

class SportsCar(Car):
    def start(self):
        print("Sports Car started")

obj = SportsCar()
obj.start()
```

Output

```
Sports Car started
```

Polymorphism

"Many forms" — the same method call behaves differently per object

Polymorphism means "many forms" and allows the same method, function or operator to behave differently depending on the object or data it works with. This flexibility helps create more reusable, maintainable and scalable code.

Real-Life Example: In a backend payment system, multiple payment options are available such as Credit Card, UPI, NetBanking and Wallet. All payment types use a common method named processPayment() but different implementations:

- Credit Card Payment: validates card, talks to bank API
- UPI Payment: redirects to UPI gateway
- Wallet Payment: checks wallet balance
- NetBanking Payment: redirects to bank login

The method name remains the same, but the action changes based on the payment type.

Polymorphism

"Many forms" — the same method call behaves differently per object

```
class Dog:
  def speak(self): return "Woof!"

class Cat:
  def speak(self): return "Meow!"

class Cow:
  def speak(self): return "Moo!"

for animal in [Dog(), Cat(), Cow()]:
  print(animal.speak())

# Woof!
# Meow!
# Moo!
```

- No shared parent required here — Python uses "duck typing": if it has speak(), it works.
- The calling code (the for loop) doesn't need to know which exact class it's dealing with.
- Makes code flexible and extensible — add a new animal class without touching the loop.

Magic (Dunder) Methods

Hook into Python's built-in syntax and functions

`__init__`	Constructor — runs on object creation
`__str__`	Defines print(obj) / str(obj) output
`__repr__`	Defines the developer-facing representation
`__len__`	Powers len(obj)
`__eq__`	Powers obj1 == obj2
`__add__`	Powers obj1 + obj2

```
class Point:
  def __init__(self, x, y):
    self.x, self.y = x, y

  def __str__(self):
    return f"({self.x}, {self.y})"

  def __add__(self, other):
    return Point(self.x + other.x,
      self.y + other.y)

p1, p2 = Point(1, 2), Point(3, 4)
print(p1 + p2)   # (4, 6)
```

Part 1 Recap — The Pillars of OOP

1

Encapsulation

Bundle data + behaviour; control access with naming conventions

2

Inheritance

Child classes reuse and extend a parent class

3

Polymorphism

Same method name, different behaviour per object

PART 2

Exception Handling

Writing Python code that fails gracefully, not catastrophically

What Can Go Wrong?

Three broad categories of error in a Python program

Syntax Errors

Code violates Python's grammar. Caught before the program even runs.

```
print("hi"
```

Runtime Errors (Exceptions)

Code is syntactically fine but fails while executing — e.g. dividing by zero.

```
10 / 0
```

Logical Errors

Program runs and gives a result — just the wrong one. No error message at all.

```
avg = total + count  (should be
/)
```

This class focuses on runtime errors — Exceptions — and how to handle them so a program can recover instead of crashing.

Common Built-in Exceptions

The ones you'll meet in almost every program

ZeroDivisionError

```
10 / 0
```

Dividing a number by zero

ValueError

```
int("abc")
```

Right type, inappropriate value

TypeError

```
"2" + 2
```

Operation on incompatible types

IndexError

```
lst[10]
```

List index out of range

KeyError

```
d["missing"]
```

Dictionary key not found

FileNotFoundError

```
open("x.txt")
```

File doesn't exist

AttributeError

```
obj.missing_attr
```

Object has no such attribute

NameError

```
print(undefined_var)
```

Variable never defined

try / except — The Basics

Wrap risky code so a failure doesn't crash the whole program

```
try:
  num = int(input("Enter a number: "))
  result = 10 / num
  print(result)
except ZeroDivisionError:
  print("Cannot divide by zero!")

print("Program continues...")

# Input 0:
# Cannot divide by zero!
# Program continues...
```

try:

Code that might raise an exception goes here.

except ExceptionType:

Runs only if that specific exception occurs.

Flow

If no error: except block is skipped entirely. If error: remaining try lines are skipped, except runs.

Program continues

Code after the try/except still runs — no crash.

Handling Multiple Exceptions

Different problems, different responses

```
try:
  num = int(input("Enter a number: "))
  result = 10 / num
except ZeroDivisionError:
  print("Cannot divide by zero!")
except ValueError:
  print("That's not a valid number!")
except (TypeError, KeyError) as e:
  print(f"Something else went wrong: {e}")
except Exception as e:
  print(f"Unexpected error: {e}")
```

- Stack specific except blocks from most specific to most general.
- A tuple (TypeError, KeyError) catches either in one block.
- except Exception as e catches almost anything else — a safety net, usually placed last.
- Python checks except blocks top-to-bottom and stops at the first match.

else and finally

Completing the try statement

```
try:
  num = int(input("Enter a number: "))
  result = 10 / num
except ZeroDivisionError:
  print("Cannot divide by zero!")
else:
  print(f"Success! Result = {result}")
finally:
  print("Execution finished.")
```

else

Runs only if the try block succeeded — no exception at all. Keeps success-path code out of the try block.

finally

Always runs — whether an exception occurred or not. Used for cleanup: closing files, releasing connections.

raise & Custom Exceptions

Signal your own error conditions

```
def withdraw(balance, amount):
  if amount > balance:
    raise ValueError("Insufficient funds")
  return balance - amount

# Custom exception class
class InsufficientFundsError(Exception):
  pass

def withdraw(balance, amount):
  if amount > balance:
    raise InsufficientFundsError(
      f"Cannot withdraw {amount}, balance is
{balance}")
```

- raise lets your own code trigger an exception when a condition you define is violated.
- Custom exceptions subclass Exception (or a built-in) to give errors meaningful, specific names.
- Callers can catch InsufficientFundsError just like any built-in exception.

Question 2

A Safe Input-Validated Calculator

- Ask the user for two numbers and an operator (+, -, *, /)
- Wrap the input conversion in try / except to catch ValueError for non-numeric input
- Catch ZeroDivisionError separately for division by zero
- Use else to print the result only on success, and finally to print "Calculation attempt complete"
- Bonus: raise a custom InvalidOperatorError if the operator isn't one of +, -, *, /

Key Takeaways

What to walk away remembering

OOP

- A class is a blueprint; an object is a specific instance
- self refers to the current object; __init__ sets it up
- Inheritance reuses code via super(); overriding customises it
- Polymorphism: same method name, different behaviour

Exceptions

- try / except stops one failure from crashing the whole program
- Catch specific exception types before a general Exception
- else runs on success only; finally always runs — great for cleanup
- raise and custom exception classes let your code signal its own errors

Questions?

Thank you — let's put OOP and exception handling to work.

Prabu | Dept. of Computer Science and Engineering | Amrita School of Engineering, Chennai